# Take-Home Assignment — Portfolio Value Service

Thanks for taking the time to do this. It's deliberately small: we expect it to take **4–5 hours at most** across the backend and a small front end, and we'd genuinely rather see a small thing you can explain deeply than a big thing you can't. There will be a 30-minute follow-up call where you walk us through what you built and why.

## AI tools are allowed and encouraged

Use Claude Code, Copilot, ChatGPT, or whatever you normally work with — we use these tools daily ourselves, and we want to see how you work with them, not how you work without them. The only rule that matters: **you own every line in the repo.** On the call we will pick parts of the code and ask you to explain what they do and why they're there. "The AI wrote that part" is not an answer we can work with, so review and understand everything you ship, exactly as you would for a production PR.

## The problem

Build a small backend service that tracks the total value of a simple investment portfolio containing both crypto and fiat currency, valued in CAD.

You'll pull live data from two free public APIs:

- **CoinGecko** for crypto prices. The keyless public tier works without any signup — try it right now: `curl "https://api.coingecko.com/api/v3/simple/price?ids=bitcoin,ethereum&vs_currencies=cad"` (Note: the base URL alone returns an "Incorrect path" error — only full endpoint paths work.) It is rate-limited to roughly 10–30 calls per minute and returns **429** when you exceed that — how you deal with this is part of the exercise. A free demo API key is available if you register; using one is fine, but your design should still assume the API can throttle or fail.
- **Frankfurter** for fiat exchange rates (ECB data, no key, no signup): `curl "https://api.frankfurter.dev/v1/latest?base=USD&symbols=CAD"` Historical rates and ranges are also available (see frankfurter.dev docs).

The two APIs have completely different response shapes and conventions. Your service is the thing that hides that mess from its own clients.

## What your service must do

1. `POST /portfolios` — create a portfolio from a list of holdings, e.g. `[{"asset": "BTC", "amount": 0.5}, {"asset": "ETH", "amount": 10}, {"asset": "USD", "amount": 2500}, {"asset": "EUR", "amount": 1000}]`. Portfolios must survive a service restart — persist them in a datastore of your choice. **Any storage is acceptable** — Postgres, Mongo, SQLite, Redis, a file — but the choice, and the shape of what you store, is a design decision we'll spend real time on in the call.
2. `GET /portfolios/:id/value` — the portfolio's current total value in CAD, plus a per-holding breakdown (amount, unit price/rate in CAD, subtotal). Crypto values come from CoinGecko, fiat from Frankfurter, normalized into one consistent response shape.
3. **Be a good citizen toward the upstreams.** If ten clients ask for the same portfolio's value in the same minute, hammering CoinGecko ten times is the wrong answer. What you do instead — and what staleness you accept in exchange — is a design decision; be ready to defend the freshness/traffic trade-off you chose. (Hint that matters: Frankfurter's rates update once per working day. Whether your design exploits that fact is interesting to us.)
4. **Handle upstream failure honestly.** If CoinGecko rate-limits or times out, the endpoint must not crash and must not silently present stale or partial data as fresh and complete. What the client sees instead — an error, a degraded response, clearly-labeled cached data — is your call to make and defend.
5. `GET /portfolios/:id/history` — record a valuation snapshot whenever a portfolio's value is computed, and return the history (timestamp + total). How you store history, and how you'd stop it growing forever, is a conversation we'll have.

Keep scope tight on the backend: no auth, no users, no deployment. A local `npm start` (or `docker compose up`) is all we need.

## The front end

This role is roughly 80% backend and 20% front end, so we want to see both. Build a **small React page** (plain React,

Vite or Next — your choice; no React Native needed) that consumes your own API. Nothing fancy is expected visually — unstyled is fine, we are not assessing CSS — but it must actually work against your running service.

It needs to do four things:

1. **Show the portfolio value and per-holding breakdown** from `GET /portfolios/:id/value`, with real loading and error states. Remember one of your upstreams is slow and rate-limited: what the user sees during a 3-second wait, and what they see when the request fails, are decisions we'll ask about.
2. **Let the user edit holdings** (add, remove, change an amount) and save them back to the API. Keep the form simple; we care about how you manage the state, not how it looks.
3. **Render the valuation history as a chart.** Use any charting library you like. The history endpoint can return a lot of points — assume it could be thousands — and the page should stay responsive. If you do something to keep it that way (downsampling, virtualization, memoization, limiting what you request), say so in DECISIONS.md, because we will ask why.
4. **Handle a stale or degraded response honestly.** If your API tells the client the crypto prices came from cache or that an upstream failed, the UI must make that visible rather than silently showing numbers as if they were fresh. How you surface it is up to you.

Be ready on the call to explain your component structure, where state lives and why, and what re-renders when the user edits a holding.

## What to hand in

A repo (GitHub link or zip) containing the code (backend and front end), a `README` with setup steps for both, and a short **DECISIONS.md**: a few paragraphs on the choices you made — storage and data model, the caching/freshness trade-off, failure handling, and on the client side your state management approach and how you kept the chart responsive — plus what you would do differently with more time. Honest beats polished; "I took a shortcut here because of the timebox, in production I'd do X" is a great sentence to find in that file.

Tests are welcome but not required. If you write any, one meaningful test (e.g., the rate-limit/failure path) is worth more than twenty trivial ones.

## The follow-up call (30 min)

You'll share your screen, run the service and the page, and walk us through both. Expect questions like "why did you store it this way," "what happens if CoinGecko 429s you right here," "walk me through this function," "what re-renders when I change this amount," and at least one small "how would you change it so that…" — the conversation is the actual assessment, and the code is the shared artifact we have it around.